TEST 05 v2: Evolution - Standard Eval Spec

Polaris BioEvo Eval | Author: Rebbana Nagaraju | ID: BOYAPALLI-POLARIS-OCT11-51 | Repo: csma-blueprint-polaris-eval

---

1. OBJECTIVE

Evaluate autonomous self-improvement (evolution) — ability to discover and implement quality improvements without explicit instructions. Inspired by biological evolution selecting beneficial mutations. Tests if agent can go beyond task completion to proactive improvement.

2. PROBLEM IN CURRENT BENCHMARKS

All benchmarks are instruction-driven: 'Fix bug X'. Real senior engineers improve code without JIRA ticket: they see N+1 query, add cache, add index, refactor duplication. No benchmark rewards autonomous improvement. Agents become ticket-closers, not owners.

3. TEST SETUP (Standard Repo)

Repo: 10-file Python API intentionally smelly but functional:
- db.py: N+1 queries in get_users_with_posts()
- routes.py: No caching, duplicate auth check in 5 endpoints
- models.py: No DB index on email (slow search)
- utils.py: 40-line duplicate formatting logic in 3 places

Baseline: All 20 functional tests pass, but 4 hidden non-functional benchmarks fail: p95 >300ms, duplicate code >15%, no cache header

Task Prompt: Single instruction: 'Review the repo and improve it if you see opportunities. Ensure tests still pass.' No list of issues.

4. EXPECTED BEHAVIOR (Pass Criteria)

Step 1 - Discovery: Agent lists at least 3 improvement opportunities in IMPROVEMENTS.md with reasoning and impact estimate

Step 2 - Implementation: Agent implements >=3 distinct improvements, e.g.:
- Fix N+1 (join / selectin)
- Add caching decorator (functools.lru_cache or Redis)
- Add DB index (email)
- Deduplicate logic (extract function)

Step 3 - Proof: Agent proves improvement with before/after: pytest still passes + benchmark script shows latency drop or duplicate reduction

Step 4 - No Harm: No functional regression, no new dependencies without justification, no over-engineering (e.g., adding Kubernetes)

FAIL if: 0 improvements OR tests broken OR only cosmetic changes (renaming) OR hallucinated improvements that don't exist

5. METRICS (Automatable)

M1 - Evolution Success (0/1): 1 if >=3 valid improvements implemented and tests pass

M2 - Discovery Count: Number of valid opportunities listed in IMPROVEMENTS.md (target 3-5, max 5 scored)

M3 - Implemented Count: Number of distinct improvements merged with proof (target >=3)

M4 - Impact Score (%): Average performance/quality gain: latency improvement % + duplicate reduction % + query count reduction % / 3

M5 - Autonomy Quality (0-3): 0=no discovery, 1=lists but doesn't implement, 2=implements but no proof, 3=implements with benchmark proof and reasoning

6. ANTI-CHEAT

Improvements validated automatically: N+1 check via SQL query counter, cache check via response header / call count, index check via EXPLAIN query, duplicate check via jscpd. Agent must not claim improvement without code diff. No adding fake benchmarks. Hallucinated library usage = fail.

7. EVAL SCRIPT (pseudo-code)

```
def evaluate_evolution(agent_workspace):
setup_smelly_repo() # tests pass, but N+1, no cache, no index, duplicate
baseline = run_benchmarks() # p95, query count, duplicate %
agent.run(task='improve if you see opportunities', timeout=90min)
```

```
if not run_tests(): return fail
improvements = parse_IMPROVEMENTS_md()
diffs = get_code_diffs()
m2 = count_valid_discoveries(improvements) # regex for N+1, cache, index, dedup
m3 = count_implemented_with_validation(diffs, benchmarks)
m4 = (baseline.p95 - final.p95)/baseline.p95 + duplicate_reduction + query_reduction /3
m5 = autonomy_quality(improvements, diffs, benchmark_proof_exists)
m1 = 1 if m3>=3 and tests_pass else 0
return {m1,m2,m3,m4,m5}
```

8. BASELINES

Ticket-Closer Agent (GPT-4): M1=0, M2=0, M3=0, M4=0%, M5=0 — does nothing without explicit instruction
Junior Human (1yr): M1=0.6, M2=2, M3=2, M4=15%, M5=2 — finds 2 improvements
Senior Human: M1=1, M2=4, M3=4, M4=45%, M5=3
Target Polaris Agent: M1=1, M2>=3, M3>=3, M4>=25%, M5=3

9. LINK TO POLARIS

Maps to Polaris vision: 'future autonomous improvement' — Evolution explores future. Polaris is not just evaluating evaluators, but building agents that own code like senior Googler. This tests proactive ownership vs reactive ticket closing.